

创建型模式 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/hml>

课程团队：刘知远 任炬 黄民烈

本讲内容提要

- 设计模式：创建型模式
- 14.1 单例（Singleton）模式
- 14.2 工厂方法（Factory Method）模式
- 14.3 抽象工厂（Abstract Factory）模式

单例模式 Singleton

例子：全局计数器

■ 一个在多个地方被调用的计数器

- ```
void func() {
 addCount();
}
```

```
int main() {
 addCount();
 func();
 cout << getCount() << endl; // 2
}
```

## ■ 如何实现addCount和getCount函数？

# 全局变量

## ■ 一种最直接的方法：

- `int count = 0;`

```
void addCount() {
 count += 1;
}
```

```
int getCount() {
 return count;
}
```

## ■ 好的设计应当避免全局变量

## ■ 用户可能访问到**count**并修改数据，不安全

# 静态方法

## ■ 定义一个类，将函数实现为静态 (**static**) 方法：

- ```
class Counter {  
    private:  
        static int count = 0;  
    public:  
        static void addCount() {  
            count += 1;  
        }  
        static int getCount() {  
            return count;  
        }  
};
```
- ```
Counter::addCount();
```
- ```
cout << Counter::getCount() << endl;
```

静态方法 & 虚函数

■如果我们有多种不同的Counter.....

- class BaseCounter {
public:
 static virtual void addCount() = 0;
 // ...
};
- class SimpleCounter : public BaseCounter {
 static int count = 0;
public:
 static void addCount() {
 count += 1;
 }
 // ...
};
- class NotSimpleCounter ...

■可以这么写吗？

静态方法 + 虚函数 = 编译错误

- 静态方法不可以是虚的！
- 静态方法只与类相关而不与实例相关，调用时必须知道类名
- 我们用静态方法实现的是：无论在何处调用，都会访问到相同的函数和变量
- 也即，类存在全局唯一的实例

单例模式

■ 所谓单例，就是只能构造一份实例的类

```
• class Counter {  
    // 显式删除拷贝构造函数与赋值操作符  
    Counter(const Counter &) = delete;  
    void operator =(const Counter &) = delete;  
  
    int count;  
    Counter() { count = 0; }  
  
    static Counter _instance; // 全局唯一的实例  
public:  
    static Counter &instance() {  
        return _instance;  
    }  
    // 成员函数而非静态方法  
    void addCount() { count += 1; }  
    int getCount() { return count; }  
};
```

单例模式

■ 调用单例

- `class Counter { ... };`
// 定义类中的静态成员，单例在此被初始化
`Counter Counter::_instance;`
- `int main() {`
// 由于删去了拷贝构造函数，必须存为引用
`Counter &c = Counter::instance();`
`c.addCount();`
`cout << c.getCount() << endl;`
`}`

惰性初始化 (Lazy Initialization)

■ 能否在使用时再构造单例实例?

- ```
class Counter {
 // ...
public:
 static Counter &instance() {
 static Counter _instance;
 return _instance;
 }
 // ...
};
```

## ■ 在第一次调用**instance**方法时才会构造单例

# 单例模式：陷阱！

- 刚才的实现真的是没问题的吗？
- 需要避免的情况：
  - 实例被重复构造
    - 由于构造函数为private，且拷贝构造函数、赋值操作符被显式删除，故无法重复构造。
  - 实例被意外删除
    - 由于返回值必须以引用形式存储（而非指针或实例），故无法被意外删除。

# 单例模式：陷阱！

- 刚才的实现真的是没问题的吗？
- 需要避免的情况：
  - 实例被重复构造
    - 由于构造函数为private，且拷贝构造函数、赋值操作符被显式删除，故无法重复构造。
  - 实例被意外删除
    - 由于返回值必须以静态变量形式存储（而非指针或实例），故无法被意外删除。

错！

# 单例模式：陷阱！

## ■ 考虑下面的代码：

- `Counter &c = Counter::instance();`  
`delete &c; // 可以成功执行!`  
`c.addCount(); // 运行时错误`

## ■ 应当把析构函数也设为**private**!

- ```
class Counter {  
    private:  
        ~Counter() {}  
    // ...  
}
```
- **error:** calling a private destructor of class 'Counter'
`delete &c;`
 [^]

单例模式 + 虚函数

■回到一开始的问题:

- ```
class BaseCounter {
 public:
 virtual void addCount() = 0;
 virtual int getCount() = 0;
};
```
- ```
class SimpleCounter : public BaseCounter {  
    // ...单例相关的一大堆逻辑...  
    int count;  
    SimpleCounter() { count = 0; }  
    public:  
        virtual void addCount() { count += 1; }  
        virtual int getCount() { return count; }  
};
```
- ```
class NotSimpleCounter : public BaseCounter ...
```

# 单例模式 + 虚函数

- ```
void doStuff(BaseCounter *counter) {  
    counter->addCount();  
    counter->addCount();  
    cout << counter->getCount() << endl;  
}
```
- ```
int main() {
 doStuff(&SimpleCounter::instance()); // 2
 doStuff(&NotSimpleCounter::instance()); // ...
 doStuff(&SimpleCounter::instance()); // 4
}
```

- 唯一的不便：单例相关的逻辑较多，有很多重复代码，却无法在基类中实现
- 为什么？因为基类不知道派生类的类别
- 能否让基类知道这一信息呢？

# “奇特的递归模板模式”

## ■ “奇特的递归模板模式”：Curiously Recurring Template Pattern (CRTP)

- `template <class Derived>` // 模板参数为派生类类型  
class Singleton {  
 Singleton(const Singleton &) = delete;  
 void operator =(const Singleton &) = delete;  
protected:  
 Singleton() {}  
 virtual ~Singleton() {}  
public:  
 static `Derived &instance()` { // 魔法在此发生  
 static `Derived _instance`;  
 return \_instance;  
 }  
};

# C RTP + 多重继承

## ■ 基于Singleton类实现计数器派生类:

- ```
class SimpleCounter : public BaseCounter,  
    public Singleton<SimpleCounter> {  
    // 友元声明是必要的，因为Singleton类需要访问派生类的  
    // 构造函数，而为了实现单例，构造函数是私有的  
    friend class Singleton<SimpleCounter>;  
    // ... 只需实现计数器逻辑即可  
}
```

■ 注意，不能直接将Singleton类的逻辑实现在BaseCounter类中，否则不存在“基类指针”

关于CRTP

- **CRTP是实现多态的另一种方式**
- **与虚函数不同，本质上实现的还是编译期多态**
- **考虑一个接受任意派生类实例的函数，在函数中调用实例的方法：**
 - 使用虚函数实现：运行时通过虚函数表寻找调用的方法
 - 使用CRTP实现：函数需要被实现为模板函数，编译时由编译器为每种被调用的派生类进行模板实例化

关于单例模式

- 单例模式是存在争议的一种设计模式

- 优点：

 - 实现似乎比较简单

 - 以相对安全的形式提供可供全局访问的数据

- 缺点：

 - 难以完全正确地实现

 - 违反单一职责原则

 - 过度使用这一方法会使得实际的依赖关系变得隐蔽

工厂方法模式

Factory Method

例子：泡茶程序

■根据客户需求提供绿茶或者红茶：

- ```
GreenTea *orderGreenTea() {
 GreenTea *greenTea = new GreenTea();
 greenTea->addIngredients(); // 加料
 greenTea->brew(); // 泡茶
 greenTea->pour(); // 装杯
 return greenTea;
}
```
- ```
BlackTea *orderBlackTea() {  
    BlackTea *blackTea = new BlackTea();  
    blackTea->addIngredients(); // 加料  
    blackTea->brew();           // 泡茶  
    blackTea->pour();           // 装杯  
    return blackTea;  
}
```

■代码存在大量冗余！

例子：泡茶程序

■ 抽出公共逻辑并简化：

- ```
Tea *orderTea(string type) {
 Tea *tea
 if (type == "GreenTea")
 tea = new GreenTea;
 else if (type == "BlackTea")
 tea = new Blacktea;
 tea->addIngredients(); // 加料
 tea->brew(); // 泡茶
 tea->pour(); // 装杯
 return tea;
}
```

## ■ 类似的构造过程可能在许多地方用到，进一步抽象为一个方法

# 工厂方法

## ■ 在Tea类中添加一个factory静态方法：

```
• class Tea {
 // ...
 public:
 static Tea *factory(string type) {
 if (type == "GreenTea")
 return new GreenTea;
 else if (type == "BlackTea")
 return new Blacktea;
 // ... 其他可能的茶叶类型
 }
};
```

## ■ 这一方法被称为“工厂方法”

# 单独的工厂类

- 当构造逻辑过于复杂，或者有必要进行分离时，可以把工厂方法放在单独的类中：

- `class TeaFactory {`  
  `public:`  
    `void setMilk(int amount) { ... }`  
    `void setSugar(int amount) { ... }`  
    `Tea *createTea(string type) {`  
      `Tea *tea = nullptr;`  
      `if (type == "GreenTea")`  
        `tea = new GreenTea;`  
      `else if (type == "BlackTea")`  
        `tea = new Blacktea;`  
      `if (milkAmount > 0) tea->addMilk(...);`  
      `if (sugarAmount > 0) tea->addSugar(...);`  
      `// ...`  
    `}`  
  `};`

# 工厂方法的用途

- 包装复杂的构造逻辑
- 为重载的构造函数提供描述性名称
- 对象构造需要用到当前函数体无法访问的信息
- 需要集中管理被构造对象的生命周期

# 工厂方法的用途

- 包装复杂的构造逻辑
- 为重载的构造函数提供描述性名称
- 对象构造需要用到当前函数体无法访问的信息
- 需要集中管理被构造对象的生命周期

# 构造函数的描述性名称

- 一些类可能具有多个重载的构造函数，可以改写为工厂方法以使用描述性的名称：

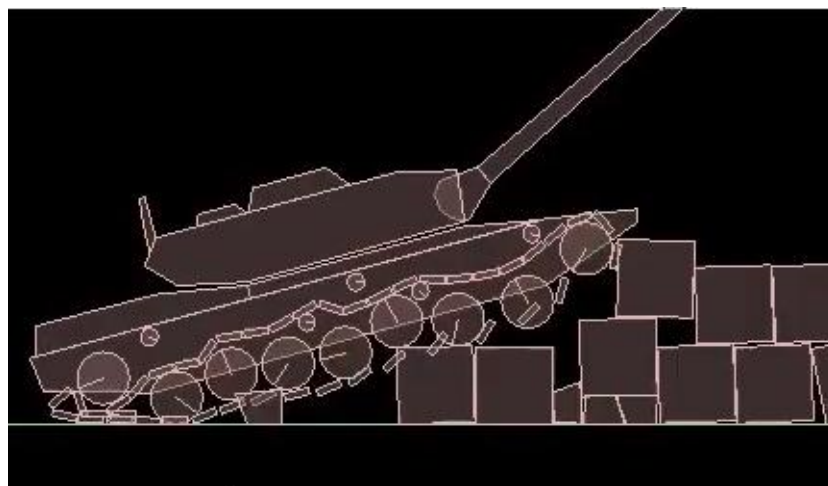
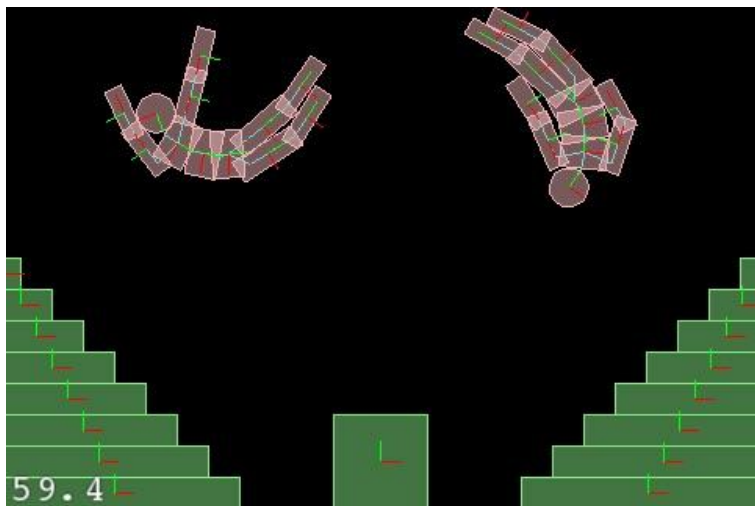
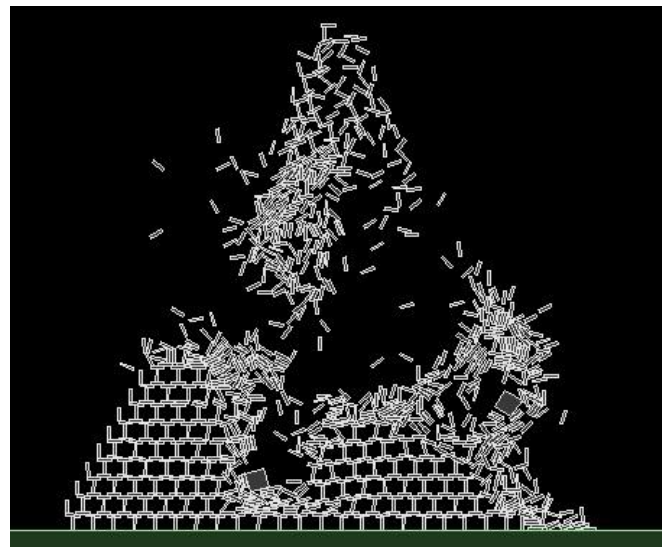
- ```
class Complex { // 复数类
    float real, imag; // 实部、虚部
    Complex(float real, float imag) { ... }
public:
    // 笛卡尔坐标
    static Complex fromCartesian(float x, float y) {
        return Complex(x, y);
    }
    // 极坐标
    static Complex fromPolar(float r, float a) {
        return Complex(r * cos(a), r * sin(a));
    }
};
```

工厂方法的用途

- 包装复杂的构造逻辑
- 为重载的构造函数提供描述性名称
- 对象构造需要用到当前函数体无法访问的信息
- 需要集中管理被构造对象的生命周期

一个现实的例子: Box2D

- **Box2D** 是一款高性能、工业级的 **2D** 物理引擎。它在处理物理对象（如刚体、碰撞体）的创建时，并没有直接让用户使用 **new** 操作符，而是采用了严格的工厂模式。



Box2D中的主要类型

■ b2World:

执行物理模拟的世界对象，包含所有需要模拟的 b2Body 对象

■ b2BodyDef:

描述一个 b2Body 对象所拥有的属性，如位置、方向、速度与角速度、是否受重力等

■ b2Body:

刚体对象，其中可以包含多个物体，但其相对位置和方向不会发生变化

b2World (物理世界)

- 管理所有刚体的"容器"
- 负责物理步进、碰撞检测等核心逻辑
- 持有内存分配器 b2BlockAllocator

基于

b2BodyDef (配置描述)

- 纯数据对象，无业务逻辑，可复用，可序列化
- 作用：解耦配置和创建

构建

b2Body (刚体实例)

- 构造函数为 private
- 只能通过 b2World::createBody 创建
- 内部持有 b2World 引用，用于内存管理

```
b2Body *body = world->createBody(&bodyDef);
```

Box2D中的工厂方法

■ 构造b2Body必须使用工厂方法，构造函数为private

- `b2World *world = new b2World(...);`
- `b2BodyDef bodyDef(...);`
- `b2Body *body = world->createBody(&bodyDef);`

■ 原因有二：

1. b2Body在构造时需要用到b2World的私有成员。
2. Box2D需要频繁申请并释放小块内存，为高效处理，Box2D自己实现了小块内存分配器b2BlockAllocator并手动管理内存。通过工厂方法，可以避免用户误用系统的内存分配机制。

■ 优势：

- 封装复杂创建逻辑（内存分配、世界注册、碰撞初始化等）
 - 保证对象状态一致性
 - 支持未来扩展（如添加新类型刚体，只需改工厂逻辑即可）
- 另外，使用b2BodyDef可以避免在函数调用中标明所有属性，也可以方便地构造多个属性相同或类似的b2Body对象

抽象工厂模式

Abstract Factory

工厂方法的局限性

- 工厂方法的目的是构造单个类的对象
- 如果我们要构造的是多个类的对象，而且有特定的组合方式呢？
- 一个很现实的例子：编译器前端
 - 编译器前端负责由代码构建统一格式的中间表示
 - 通常来说，需要三个步骤：语法分析、语义分析、中间代码生成
 - 对于每种语言来说，每个步骤的实现不同，同一种语言应该使用同一种实现
 - 编译器前端的框架是相同的，只需要替换三个步骤的不同实现
 - 不妨假设有两种语言：C++和Java

编译器前端设计

■ 三个步骤的类:

- 语法分析:
Lexer
CppLexer
JavaLexer
- 语义分析:
Parser
CppParser
JavaParser
- 中间代码生成:
Generator
CppGenerator
JavaGenerator

■ 框架的实现:

```
class Compiler {  
    string type;  
public:  
    Compiler(string type) {  
        this->type = type;  
    }  
    LexResult *lex(Code *input) {  
        Lexer *lexer;  
        if (type == "cpp")  
            lexer = new CppLexer;  
        else if (type == "java")  
            lexer = new JavaLexer;  
        return lexer->lex(input);  
    }  
    ParseResult *parse(LexResult *input) {  
        Parser *parser;  
        if (type == "cpp")  
            parser = new CppParser;  
        else if (type == "java")  
            parser = new JavaParser;  
        return parser->parse(input);  
    };  
    // ...  
};
```

框架的实现细节

■ 框架如此实现有其合理性：

1. 分阶段进行可以解耦合，方便测试
2. 延迟初始化可以缩短Compiler类的构造时间

■ 然而当前实现有过多代码重复，如果要添加新语言支持，需要在每个步骤中加一条if分支

■ 我们如何在当前框架下减少代码重复？

添加一层抽象

■ 设计一个基类，抽象同一语言所需的所有步骤：

- class **AbstractFactory** {
 public:
 virtual Lexer *createLexer();
 virtual Parser *createParser();
 virtual Generator *createGenerator();
};
- class CppFactory : public AbstractFactory {
 public:
 Lexer *createLexer() {
 return new CppLexer; }
 Parser *createParser() {
 return new CppParser; }
 Generator *createGenerator() {
 return new CppGenerator; }
};
- class JavaFactory : public AbstractFactory ...

修改框架实现

■ 将刚才的基类运用到**Compiler**类中：

- ```
class Compiler {
 AbstractFactory *factory;
 Compiler(AbstractFactory *factory) {
 this->factory = factory;
 }
 LexResult *lex(Code *input) {
 return factory->createLexer()->lex(input);
 }
 ParseResult *parse(LexResult *input) {
 return factory->createParser()->parse(input);
 }
 // ...
};
```

# 抽象工厂模式

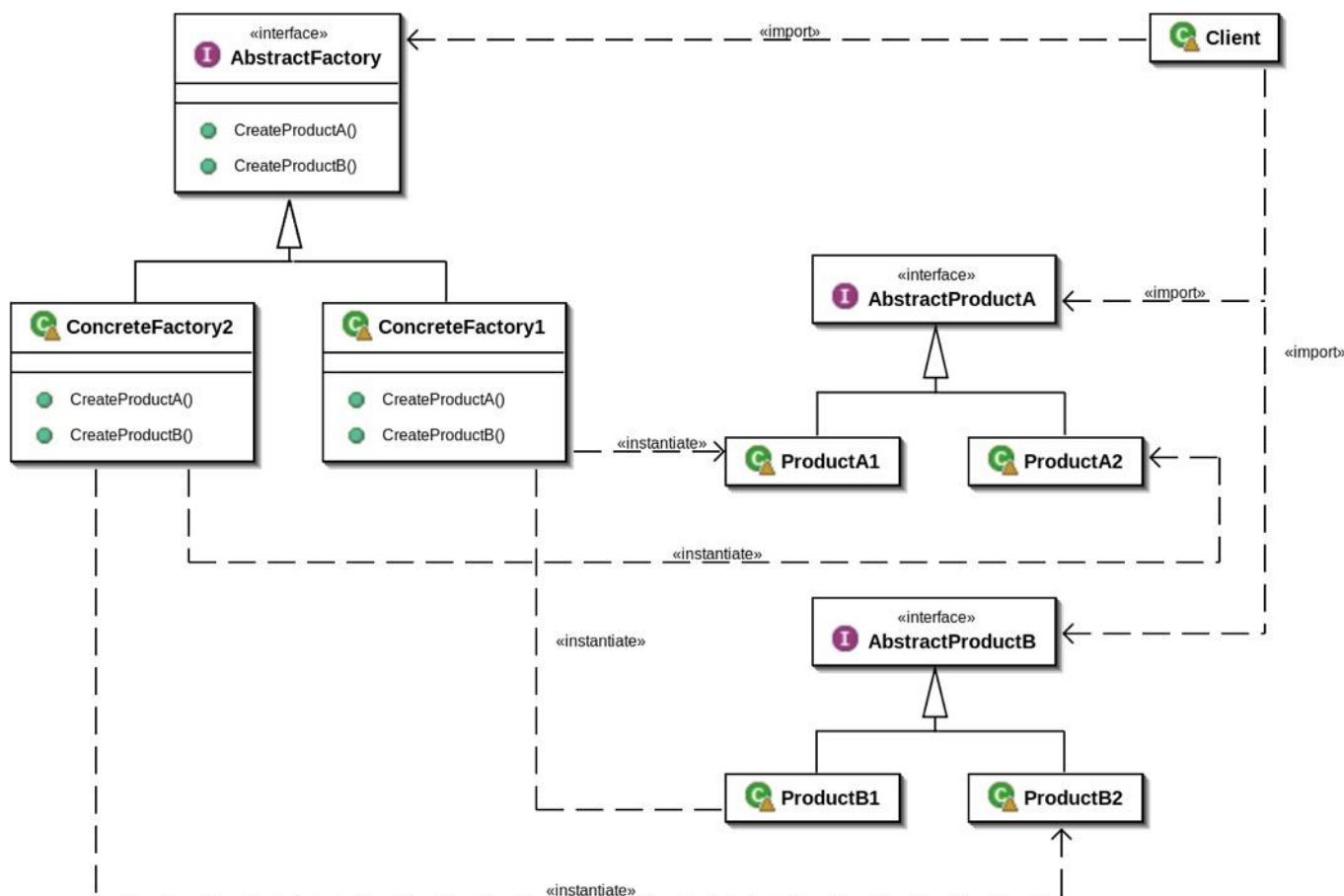
## ■ 将刚才的类对应到术语：

- 抽象工厂：AbstractFactory
- 具体（Concrete）工厂：CppFactory、JavaFactory
- 抽象产品：Lexer、Parser、Generator
- 具体产品：CppLexer、JavaParser、.....

- 抽象工厂模式即，将同类的具体产品包装为一个具体工厂，以抽象工厂的形式呈现给上层代码；  
上层代码只需要关心抽象工厂和抽象产品，而不需要知道具体的工厂和产品是哪些。

# 抽象工厂模式

■ 抽象工厂模式的UML图如下：



# 设计模式总结

# 设计模式回顾

## ■ 行为型模式 (Behavioral Patterns)

- 关注对象行为功能上的抽象，提升对象在行为功能上的可扩展性，能以最少的代码变动完成功能的增减
- 常用于描述对类和对象的交互与职责分配

### 行为型模式

- **模板方法模式**：定义算法骨架，将具体步骤的实现放到子类中实现。可以在不改变算法流程的情况下，自定义某些步骤
- **策略模式**：定义一类算法，将每个算法分别封装，不同算法可以相互替换
- **迭代器模式**：用于遍历数据集合（数组、链表、树、图等），解耦算法与数据访问

# 设计模式回顾

## ■ 结构型模式 (Structural Patterns)

- 关注对象之间结构关系上的抽象，从而提升对象结构的可维护性、代码的健壮性，**能在结构层面上尽可能解耦合**
- 常用于处理类和对象的组合关系

### 结构型模式

- **适配器模式**：将不兼容的接口转换为可兼容的接口
- **代理/委托模式**：在不改变原始类接口的条件下，为原始类定义一个代理类，增加控制访问
- **装饰器模式**：用组合来替代继承，给原始类添加增强功能

# 设计模式回顾

## ■ 创建型模式 (Creational Patterns)

- 将对象的创建与使用进行划分，从而规避复杂对象创建带来的资源消耗，能以简短的代码完成对象的高效创建
- 用于对象的创建

### 创建型模式

- **单例模式**：用来创建全局唯一的对象
- **工厂方法模式**：用来创建不同但相关类型的对象，由给定的参数来决定创建哪种类型的对象
- **抽象工厂模式**：提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们具体的类

# 设计原则

## ■ 开闭原则

- 一个软件实体，比如类，模块，函数应该对扩展开放，对修改关闭
- 最基础的设计原则

## ■ 单一职责原则

- 每个类应该只有一个职责，只有一个原因可以引起它的改变
- 例如：迭代器模式使得数据结构与算法分离；可视化程序设计中页面与逻辑分离

## ■ 里氏代换原则

- 只要父类出现的地方子类就可以出现，即子类尽量不要修改父类的数据与方法，实现基类代码的充分复用

# 设计原则

## ■ 依赖倒转原则

- 要依赖于抽象，不要依赖于具体。针对接口编程，而不是针对实现编程。具体而言就是上层模块不应该依赖底层模块，使用接口和抽象类指定好规范，剩下的具体细节由实现类来完成
- 例如：策略模式/模板方法模式不依赖于具体的策略实现，只依赖于抽象

## ■ 接口隔离原则

- 不要建立臃肿庞大的接口。即接口尽量细化的同时接口中的方法尽量少
- 功能拆分粒度太小，将使得类、接口的数量过多；功能拆分粒度太大，将使得类之间耦合度高，程序不灵活

# 设计原则

## ■ 迪米特原则

- 最少知道原则，一个对象应该对其他对象有最少的了解，使得功能模块相对独立

## ■ 合成复用原则

- 合成复用原则就是指在一个新的对象里通过关联关系（包括组合关系）来使用一些已有的对象，使之成为新对象的一部分；新对象通过委派调用已有对象的方法达到复用其已有功能的目的
- 即在实现扩展类功能时，优先考虑使用组合而不是继承；如需要使用继承，则遵守里氏代换原则

- 在程序设计中尽量遵循七大原则，但也需根据实际情况调整，切勿滥用设计模式使得代码过度冗余

结束